SDC26

Technical Report

Team ToBeDefined

Swarm Drone Challenge 2026 — ILA Berlin, 11 June 2026

Abstract

This report describes the technical solution developed by Team ToBeDefined for the Swarm Drone Challenge 2026. Each Parrot Anafi flies under an on-board mission engine driven by ArUco-only positioning (no GPS), a domain-specific mission-scripting language, and a shared command-and-control (C2) overlay that coordinates the swarm across the 20 m × 10 m arena. The design prioritises drift-free, vision-relative homing over absolute world-frame steering, uses bounded closed-loop primitives for repeatable manoeuvres, and exposes a transparent rule-based strategy layer that translates the game's scoring tiers into per-drone role and target assignments. The same code runs the Parrot Sphinx simulator and the real flight controllers, so every change is validated against identical mission scripts and telemetry contracts before flying hardware. The stack is roughly 90,000 lines of Python and is open-sourced in line with the competition's encouragement of shared code.

1. Introduction

The Swarm Drone Challenge 2026 (SDC26) pits two teams of up to five Parrot Anafi drones against each other in a 20 m × 10 m indoor arena. Each side defends three target boxes in its home zone and attempts to flip the three enemy boxes by hovering over them, within a 1–2 m capture band, for at least two seconds. The boxes carry DICT_4X4_50 ArUco markers whose encoding — leading digit 4 for red, 3 for blue, indicates the owner; trailing digit is the box ID 1–6 — flips on capture. Points accrue per attempt at three tiers (1, 5, or 10), with an instant-win Special Manoeuvre for controlling all six boxes for five continuous seconds. A match lasts ten minutes; GPS is unavailable in the indoor venue, so all positioning is derived from on-board cameras and inertial sensors.

Team ToBeDefined attacks the problem with a single, uniform software stack that runs both on each drone's flight controller (an x86 Linux companion computer flying the Anafi via Parrot Olympe) and on a lightweight central C2 overlay. The same code base powers the Parrot Sphinx simulator, which stands in for the real flight controllers during development; a single operator switch (by IP) re-points any drone from the simulator to a real controller, and a second switch hot-swaps the whole stack between the competition arena and a smaller testing arena — live, with no restart.

2. System Architecture

Three process types form the runtime. **marker_mission** (port 8080) is the per-drone flight controller: it owns the Olympe connection, runs the ArUco vision loop, executes the mission-scripting language, and exposes a small HTTP API for telemetry, RC, and mission control. **marker_mission_c2** (port 8090) is the fleet layer: it polls every flight controller, proxies operator commands, and serves the live state UI. The **strategy** servers (ports 8091 red / 8092 blue) sit on top of the C2 and own slot tracking, the per-drone role assignment, and the rule-based planner of Section 5. All inter-process traffic is HTTP/JSON, and the whole fleet is joined into a single Tailscale tailnet so the operator UIs are reachable by MagicDNS name while remaining cleanly air-gapped from the public internet.

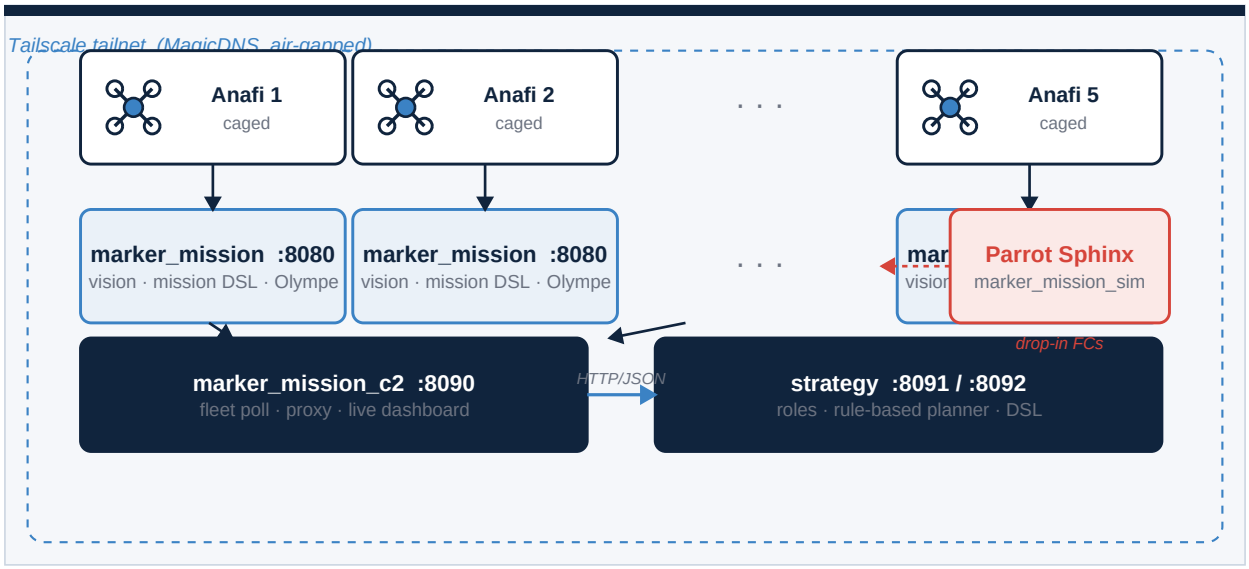

Figure 1 — Runtime architecture. Each caged Anafi is flown by its own marker_mission flight controller; the C2 aggregates the fleet and the per-team strategy servers issue role and target assignments. The Sphinx simulator is a drop-in replacement for the real flight controllers.

Mission behaviour is expressed in a small text-based scripting language — the *mission DSL* — parsed and executed by the flight controller. Its primitives cover discrete IMU closed-loop moves (`FB_IMU`, `LR_IMU`, `UD_IMU`, `YAW_IMU`), absolute heading holds (`YAW`), open-loop stick windows (`FB_RC`, `UD_RC`), vision homing (`APPROACH`, `FB_BRAKE`), holds (`HOOVER`), and coarse world-frame pre-positioning (`TO`). Each step compiles to a state-machine phase that drives a closed-loop RC stream at the configured control rate (10 Hz), gives vision its own thread, and writes a structured per-tick log consumed by an offline replay viewer for tuning.

3. Localisation

The arena provides eight pillars carrying sixteen ArUco markers — an upper marker at 4 m and a lower marker at 2 m per pillar; IDs 1–8 upper, 9–16 lower, 0.5 × 0.5 m, DICT_4X4_50. Each flight controller's vision loop runs OpenCV ArUco detection on the on-board camera and computes a per-marker pose with the IPPE_SQUARE planar-pose algorithm. Planar PnP returns two equally-valid solutions related by a reflection about the marker plane; the wrong branch places the drone on the far side of the marker, so resolving it correctly is the crux of the pipeline (Figure 2). We disambiguate with two independent priors.

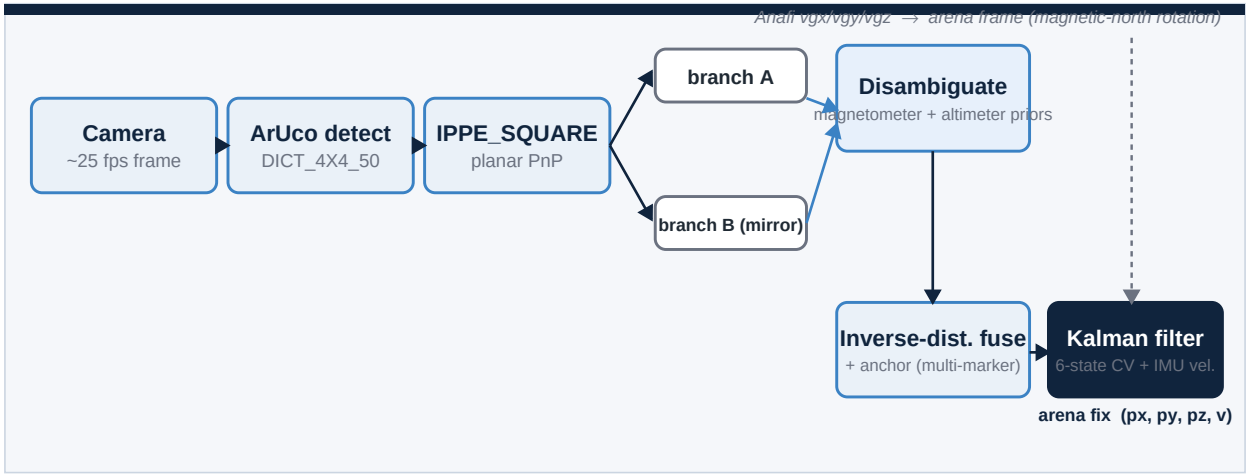

Figure 2 — Localisation pipeline. The IPPE planar-pose ambiguity is resolved by a magnetometer prior (works on the ground too) and an altimeter prior; multi-marker fixes are inverse-distance fused around a remembered anchor, then smoothed by a six-state constant-velocity Kalman filter that also ingests Anafi body velocity.

3.1 Magnetometer and altimeter priors

The arena's magnetic-north heading is calibrated once and stored on the active arena configuration. The two IPPE branches imply two candidate drone headings; the branch whose heading agrees with the live magnetometer reading within a configurable slack (default 12°) is accepted. Because this test needs no altitude, it gives a clean fix even on the ground at take-off. When the drone is airborne, the Anafi's downward ultrasound altimeter provides a second, independent check on the vertical component of each branch. We require agreement from at least one prior; if only one marker is visible and the magnetometer is uncalibrated, the fix is rejected and the previous estimate is allowed to decay rather than jump to a mirrored pose.

3.2 Kalman filter and multi-marker anchor

Raw per-tick fixes are fused with Anafi-reported body velocities in a six-state constant-velocity Kalman filter whose state is `[px, py, pz, vx, vy, vz]` in metres and metres per second. Two heterogeneous updates feed it: an intermittent, accurate position update from the aggregated ArUco fix, and a fast, every-tick velocity update from the Anafi `vgx/vgy/vgz` telemetry rotated into the arena frame using the calibrated magnetic-north offset. The filter both smooths measurement jitter and supplies the velocity estimate the controller uses to damp lateral oscillation during APPROACH. When several reference markers are visible at once, their world positions are weighted by inverse distance and fused into one arena fix; the dominant contributor is remembered as the *anchor* across frames so a noisy single-marker reading cannot drag the estimate to the opposite side of the field.

3.3 No GPS, and a hard-won failure mode

No GPS information is used anywhere in the live positioning or control path, as required by §4.4 of the regulations; GPS appears only as an off-line ground-truth reference inside the simulator during tuning. In metal-rich indoor spaces the magnetometer can still be unreliable, and we learned the hard way that an active reverse-brake wall guard acting on a mirrored single-marker fix will drive the drone *into* the wall it appears to be near, at saturated stick. The mitigation, now the standing design rule across the stack, is to never steer open-loop on an absolute fix: precise motion is always either a bounded IMU step or a vision-relative APPROACH onto a marker the camera can actually see (Section 4), which is correct regardless of which IPPE branch the world-frame estimator chose.

4. Flight Control and Trajectory Planning

The Anafi exposes both a piloting (PCMD) interface for continuous roll/pitch/yaw/throttle commands and a discrete move-by interface that closes a position loop on the drone's own IMU and optical flow. Our mission engine treats these as complementary primitive families. **IMU primitives** (`FB_IMU`, `LR_IMU`, `UD_IMU`, `YAW_IMU`) execute an exact bounded displacement and stop. **RC primitives** (`FB_RC`, `UD_RC`) pin a stick value for a duration and then auto-brake. **Vision primitives** (`APPROACH`, `FB_BRAKE`) home on a named ArUco marker's measured bearing and distance and are drift-free regardless of the world-frame fix. An absolute `YAW` primitive holds a fixed arena heading so a manoeuvre never inherits a stale heading from the previous leg.

4.1 The canonical attack manoeuvre

Every attack run is generated programmatically by the strategy server for the selected target slot and composes to the pattern in Figure 3. The attacker climbs above the 0.73 m boxes, vision-homes onto the enemy face to a 1 m stand-off, rises into the 1–2 m capture band, takes one bounded `FB_IMU` step sized to the stand-off so it stops over the box centre, and hovers for at least two seconds to flip the marker. It then climbs, turns, and vision-homes onto its own back-wall marker to return — never an open-loop cruise back across the field.

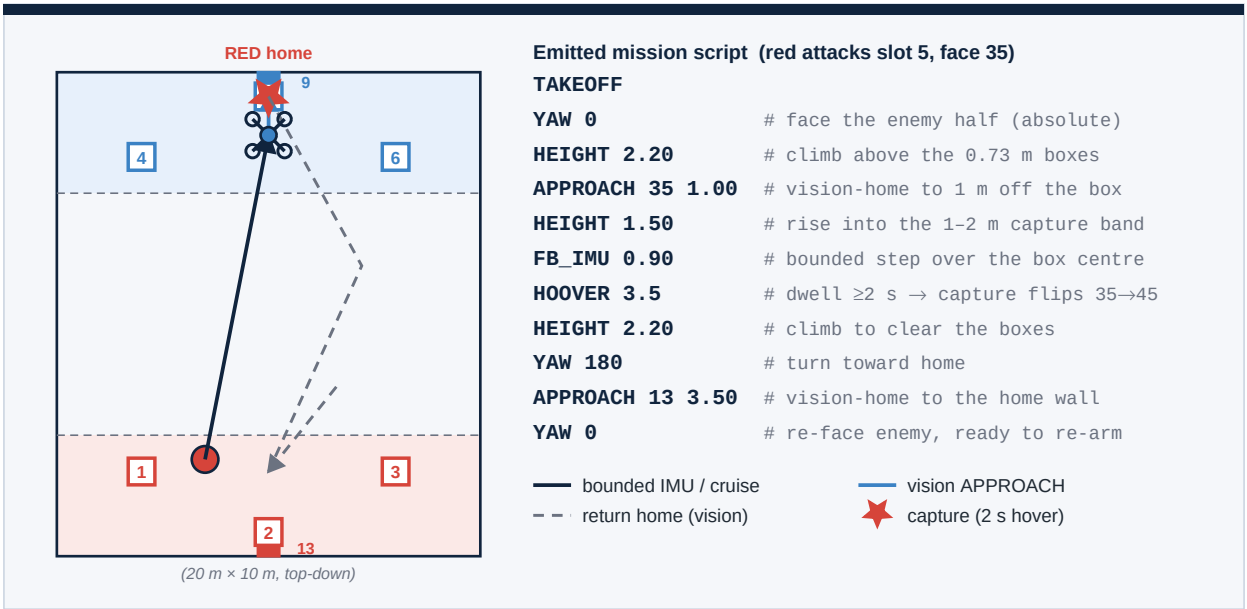

Figure 3 — The vision-relative attack manoeuvre and the exact mission script the strategy server emits. Solid navy = bounded IMU / cruise; blue = vision APPROACH; dashed = vision-homed return; star = the 2-second capture hover.

This composition reflects three hard-won lessons. First, every horizontal displacement is either a bounded IMU step or a vision-homed APPROACH — never an open-loop RC cruise of non-trivial distance — because forward pitch tilts the camera down and hides the destination marker, so a vision-tripped brake never fires and the drone reaches its timeout at full stick. Second, the over-the-box translation is a bounded `FB_IMU` sized to the APPROACH stand-off, so even with a noisy fix the drone stops above the target rather than continuing into the wall behind it. Third, the turns use the IMU-closed-loop heading holds (`YAW` / `YAW_IMU`) rather than a yaw-rate stick, which rolled off in timing tests and sent the next leg sideways. In the home zone the drone always flies higher than the 0.73 m open boxes and only descends into the capture band directly over a target.

4.2 Safety layers

Several overlays guard the runtime regardless of which script is loaded. A hard altitude ceiling clamps the throttle channel above a configurable height. A C2 watchdog forces an automatic land if the operator heartbeat goes silent for longer than the regulation's two-second threshold (§3.3). A master kill-all lands the entire fleet from a single key-press and is the responsibility of the team's Safety Officer (§2.2). Each drone in flight is shrouded in a protective cage as recommended in §3.2. The simulator enforces the same arena bounds, so a script that would drive into a wall is caught in simulation before it ever reaches hardware.

5. Swarm Strategy

Coordination is handled by a small, deterministic **planner** that runs each tick on the strategy server. It is deliberately rule-based rather than learned: every tick it reads each drone's status (connection, battery, magnetometer calibration, mission phase, position) and the live slot map (the holder of each of the six boxes and how recently it was observed), then emits a (role, target) assignment per drone. The planner is a strict priority ladder — the first applicable rule wins — which makes its behaviour transparent and reproducible, a property we value precisely because the code is open for inspection (Figure 4).

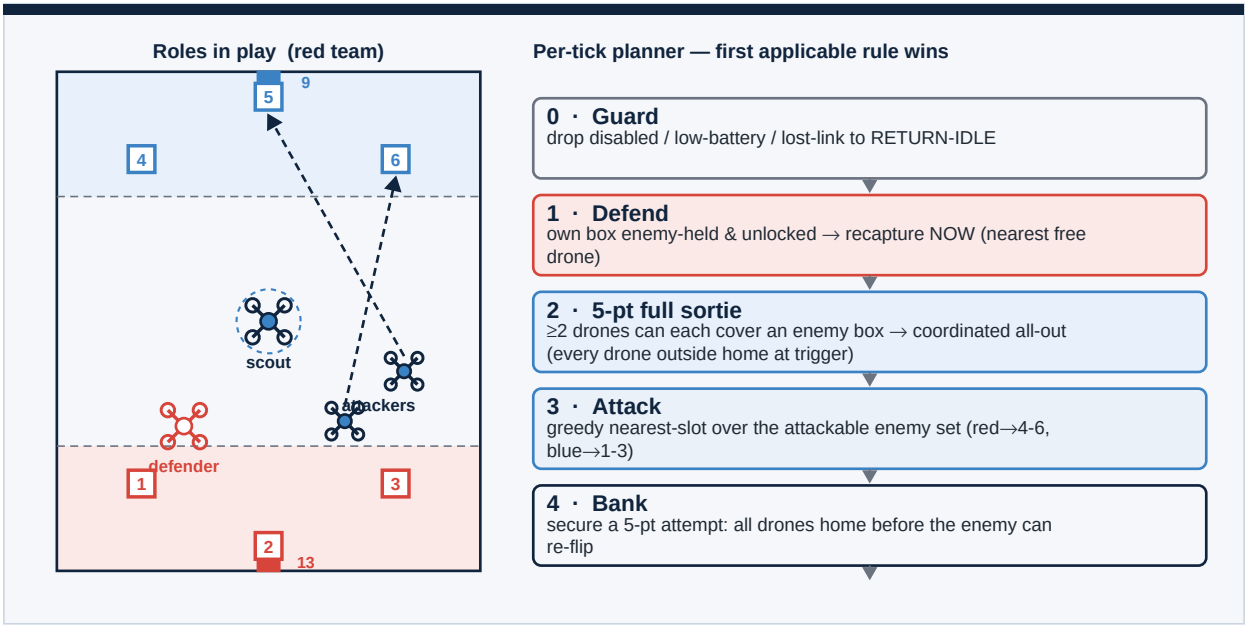

Figure 4 — Left: a typical role mix — a scout refreshing the slot map from the centre, two attackers vectored at enemy boxes, and a defender holding in the neutral zone. Right: the per-tick planner ladder; the first applicable rule wins.

The ladder encodes the regulations' scoring tiers directly. Recapturing an enemy-held box inside our own home zone scores zero under the anti-farming rule of §1.4.3, so the planner treats a home recapture as defence, not points. Rule 1 (defend) is top-priority and immediate: the moment one of our boxes shows the enemy colour, the nearest free drone breaks off to re-flip it. Rule 2 is the five-point full sortie — when at least two drones can each cover a different enemy box, the planner schedules a coordinated all-out so that every team drone is outside the home zone at the moment of capture, which the regulations reward with the coordination bonus; the team then BANKs (Rule 4), returning home together before the enemy can re-flip, to secure the attempt. Rule 3 is the fallback: a greedy nearest-slot assignment over the attackable enemy set. The Special Manoeuvre — all six boxes for five seconds — is treated as a terminal objective and short-circuits selection when within reach.

5.1 Roles, dedup, and asymmetric attack

The planner assigns one of four roles per drone: **attacker** (drive a capture or recapture), **scout** (rotate in place at the arena centre to refresh stale slots so the map stays current), **defender** (protect our home boxes), and **idle/return** (low battery, lost link, or no useful work). Assignments are de-duplicated through three sets maintained each tick — slots already *taken* by a plan this tick, slots an in-flight drone is already *targeting*, and slots inside the five-second post-capture *lock* — so two drones never converge on the same box. The attack set is asymmetric per team: red drones may only target boxes 4–6 (the blue zone) and blue drones boxes 1–3, and the planner rejects any operator override that violates the constraint.

5.2 The defender

The defender does not camp over its boxes (which would risk the §1.3 dead-drone-over-target rule and forfeit the coordination bonus). Instead it hovers at a fixed station just inside the neutral zone, facing its own back wall and holding position by a vision APPROACH on the back-wall marker — no absolute world steering. Because the slot map is shared, it detects a lost box itself the instant the marker flips and dashes in to recapture, then returns to its station. This keeps it both outside the home zone (so the team's five-point attempts stay valid) and as close as legally possible to the boxes it protects.

5.3 Open source and reproducibility

The entire stack is open-sourced at `github.com/otiedemann/sdc-tobe` — roughly 90,000 lines of Python spanning the flight controller, the C2, the strategy layer, and the simulator. The flight controller, C2, and strategy share a single configuration and virtual environment; the mission DSL ships with a test harness; and every flight is recorded into a per-tick log plus an annotated video that the in-browser replay viewer plays back synchronously, so any result in this report can be reproduced from the repository.

6. Conclusion

One design philosophy runs through every layer of the stack: prefer drift-free, vision-relative homing and bounded closed-loop motion over absolute world-frame steering and open-loop cruises. That choice is forced by the realities of ArUco-only indoor positioning, and it is what lets the swarm fly safely and repeatably. The remaining work — a position-sanity gate that lets the active wall guard be re-enabled once a fix is trusted, and further tuning of the coordinated five-point sortie — is scoped and queued, and will be delivered for the final at ILA Berlin on 11 June 2026.